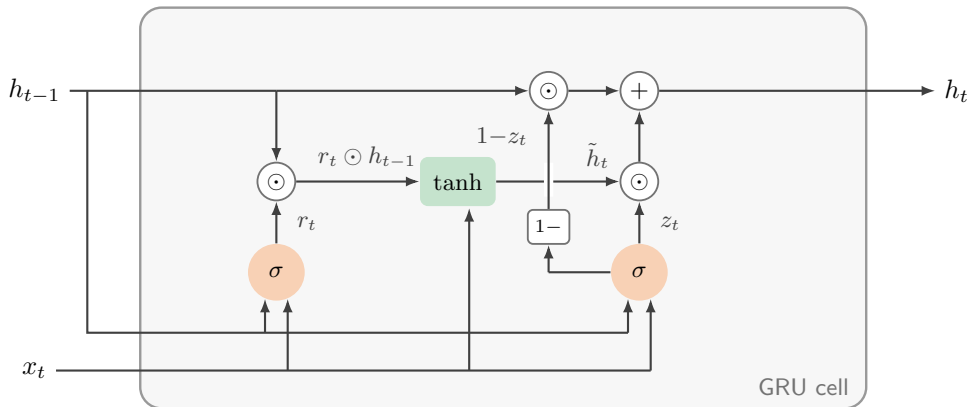


One GRU cell: two gates decide how much of the past survives



$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

reset gate: how much of the old state the candidate may read

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

update gate: how much of the state is overwritten this step

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

candidate: the state the input alone would propose

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

new state: a per-unit blend of keep and replace

σ : logistic in $(0, 1)$ $\odot$: element-wise product W, U, b : the learned weights θ

In the trial-level tiny RNN: $x_t = (a_{t-1}, s_{t-1}, r_{t-1}, s_t, \text{cue}_t)$ and h_t has $d = 1-4$ units. With $d = 1$ the cell has one reset value, one update value and one state, so the whole dynamics can be plotted.

What the gates mean here: $z_t \rightarrow 0$ keeps the criterion where it was (long memory, the Ji-An reward-learning regime); $z_t \rightarrow 1$ rewrites it from the current trial (short memory, the regime the weak sequential effects predict). $r_t \rightarrow 0$ lets the candidate ignore the past, which is how a rule switch can reset the state in one trial.

In the motor network: same cell with $x(t) = (b(t), \text{dir}, \text{go})$ every 10 ms and $M \approx 32$ units; the gates let it hold the pre-go position and then release the movement.

Why a GRU and not a plain RNN: a plain unit is $h_t = \tanh(Wx_t + Uh_{t-1} + b)$, so keeping a value and replacing it both go through the same weights; with one or two units it cannot do both well. The gates give each unit an explicit keep/replace switch that depends on the input, which is what lets a single unit hold a criterion across trials and still reset it after an error or a cue.

Cost: three weight sets instead of one. With $d = 2$ and five inputs that is about 50 parameters, still far below the 3800 trials available.